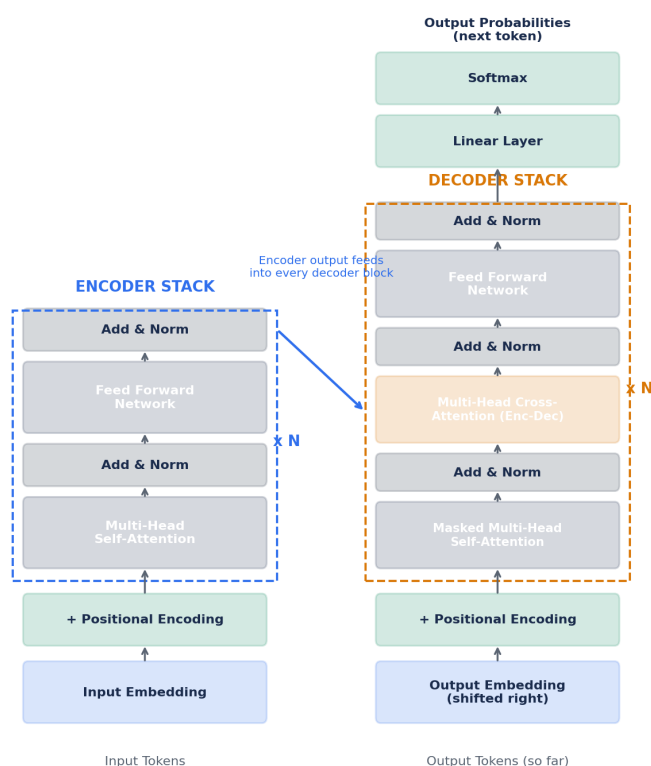


The Complete Guide to Transformers

From ANN, CNN and RNN basics to the full Transformer architecture -- Encoder, Decoder, Self-Attention, Multi-Head Attention, Positional Encoding and everything in between. Written in simple English, with diagrams for every major concept.



Contents

PART 1: THE FOUNDATIONS -- ANN, CNN, RNN

- 1. What is a Neural Network?**
- 2. ANN -- Artificial Neural Network**
- 3. CNN -- Convolutional Neural Network**
- 4. RNN -- Recurrent Neural Network**

PART 2: TRANSFORMERS -- FULL ARCHITECTURE

- 5. Why Transformers Were Invented**
- 6. What is a Transformer? -- The Big Picture**
- 7. The Full Transformer Architecture**
- 8. Input Embeddings**
- 9. Positional Encoding**
- 10. Self-Attention -- The Heart of the Transformer**
- 11. Multi-Head Attention**
- 12. Add & Norm -- Residual Connections + Layer Normalization**
- 13. The Feed-Forward Network (FFN)**
- 14. Putting It Together: The Encoder Block**
- 15. Putting It Together: The Decoder Block**
- 16. The Final Output Layer**
- 17. How Training Works**
- 18. Why Transformers Are So Powerful**
- 19. Famous Transformer-Based Models**
- 20. Applications of Transformers**
- 21. Quick Revision Sheet**
- 22. Simple Glossary**
 - 22a. Tokenization -- How Text Becomes Tokens**
 - 22b. Model Size -- Parameters, d_{model} , and Scaling**

PART 3: COMPARISON, WORKED EXAMPLE & INTERVIEW PREP

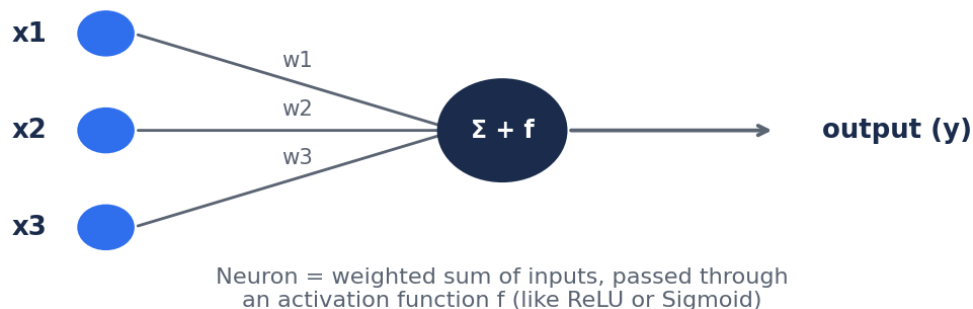
- 23. ANN vs CNN vs RNN vs Transformer -- Side by Side**
- 24. Worked Example: Self-Attention With Real Numbers**
- 25. Frequently Asked Interview Questions**
- 26. Final Word**

PART 1: THE FOUNDATIONS -- ANN, CNN, RNN

1. What is a Neural Network?

A neural network is a computer program that learns patterns from data, in a way loosely inspired by how neurons in the human brain work. Instead of writing exact rules by hand (like 'if temperature > 30, say it's hot'), we show the network many examples, and it figures out the rules on its own by adjusting internal numbers called weights.

Every neural network is built from small units called neurons (or nodes). Each neuron takes some numbers as input, multiplies them by weights, adds them up, and passes the result through a small function called an activation function. This decides how much that neuron 'fires' or contributes to the next step.



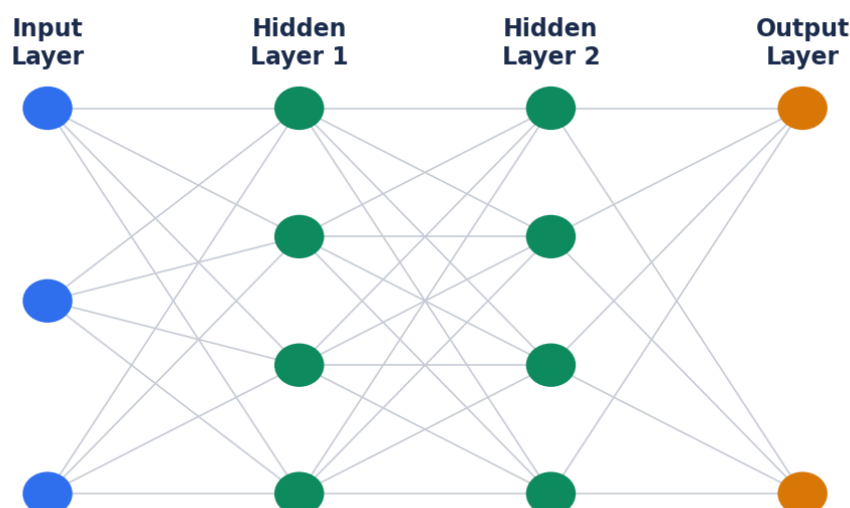
A single artificial neuron: inputs are weighted, summed, and passed through an activation function.

On its own, one neuron cannot do much. The real power comes from connecting thousands or millions of neurons together in layers. This is where ANN, CNN, and RNN come in -- they are just different ways of arranging and connecting neurons, each suited to a different kind of problem.

2. ANN -- Artificial Neural Network

What it is

ANN, sometimes called a Feedforward Neural Network or Multi-Layer Perceptron (MLP), is the most basic type of neural network. Information flows in one direction only: from the input layer, through one or more hidden layers, to the output layer. There are no loops and no memory of previous inputs.



A simple ANN: an input layer, two hidden layers, and an output layer, fully connected.

Structure

- Input Layer -- receives the raw data (numbers representing your problem, like pixel values or a table row).
- Hidden Layer(s) -- do the actual 'thinking'. Each neuron combines inputs from the previous layer and applies an activation function.
- Output Layer -- gives the final answer (like a class label, or a predicted number).
- Weights -- numbers on every connection that the network adjusts during training.
- Activation Function -- adds non-linearity so the network can learn complex patterns, not just straight lines. Common ones: ReLU, Sigmoid, Tanh.

How it learns

Training happens through a process called backpropagation. The network makes a guess, compares it to the correct answer using a loss function (a number that says how wrong the guess was), and then adjusts every weight slightly to reduce that error. This is repeated thousands of times until the network gets good at the task.

Where ANN is used

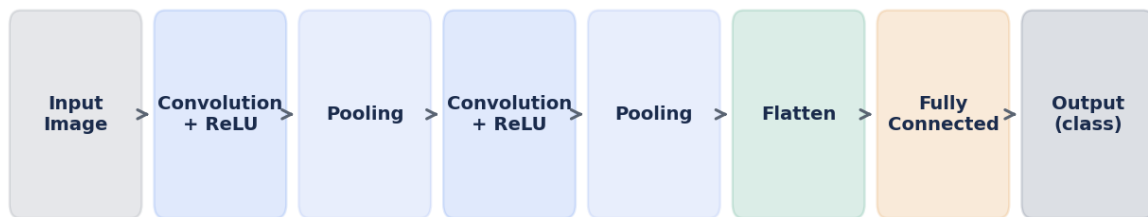
- Simple classification problems (spam vs not spam)
- Predicting a number from structured/tabular data (like predicting house prices)
- Basic pattern recognition tasks

Note: Limitation: A plain ANN treats every input independently and has no idea about spatial structure (like an image) or sequence/order (like a sentence). That's exactly why CNN and RNN were invented.

3. CNN -- Convolutional Neural Network

What it is

CNN is a type of neural network designed specifically to understand images (though it's used for other grid-like data too). Instead of connecting every input pixel to every neuron (which would be huge and wasteful), a CNN uses small filters that slide across the image, looking for patterns like edges, corners, textures, and eventually whole shapes and objects.



A typical CNN pipeline: convolution and pooling layers extract features, then fully connected layers classify.

Structure

- Convolution Layer -- small filters (like 3x3 grids of numbers) slide over the image and detect local patterns such as edges or textures.
- Activation (ReLU) -- applied after convolution to add non-linearity, same idea as in ANN.
- Pooling Layer -- shrinks the image (e.g. takes the maximum value in each small region) to reduce size and keep only the most important information. This also makes the network less sensitive to small shifts in the image.
- Repeat -- convolution and pooling layers are usually stacked several times, so early layers detect simple patterns (edges) and later layers detect complex ones (faces, objects).
- Flatten + Fully Connected Layer -- after enough features are extracted, the data is flattened into a single list of numbers and fed into a normal ANN-style layer to make the final prediction.

Why CNN works so well for images

- Parameter sharing -- the same small filter is reused across the whole image, so the network needs far fewer weights than a plain ANN would.
- Local connectivity -- each neuron only looks at a small patch of the image at a time, matching how real-world visual patterns are local.

- Translation invariance -- pooling helps the network recognize an object even if it moves slightly within the image.

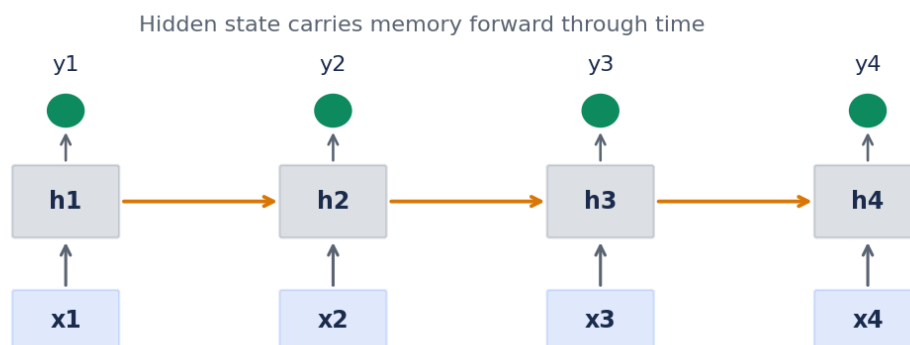
Where CNN is used

- Image classification (is this a cat or a dog?)
- Object detection (self-driving cars spotting pedestrians)
- Medical image analysis (detecting tumors in scans)
- Face recognition

4. RNN -- Recurrent Neural Network

What it is

RNN is designed for sequential data -- data where order matters, like sentences, speech, or stock prices over time. Unlike ANN and CNN, an RNN has a form of memory: it processes one item at a time (like one word at a time) and carries forward a 'hidden state' that summarizes everything it has seen so far.



An RNN 'unrolled' across time steps. The hidden state carries information from earlier words forward.

Structure

- Input at each time step ($x_1, x_2, x_3, \dots$) -- for example, one word of a sentence at a time.
- Hidden State ($h_1, h_2, h_3, \dots$) -- a running summary of everything seen so far, updated at every step using both the current input and the previous hidden state.
- Output at each time step ($y_1, y_2, y_3, \dots$) -- optional, depending on the task (e.g. predicting the next word).
- The same weights are reused at every time step -- this is what lets an RNN handle sequences of any length.

The big problem with RNNs

- Vanishing gradients -- when sequences are long, the signal used to update weights during training gets weaker and weaker as it travels back through time, so the network struggles to remember things from far back (like the start of a long paragraph).
- Sequential bottleneck -- an RNN must process word 1, then word 2, then word 3, one after another. It cannot process the whole sentence in parallel, which makes training slow on modern hardware (GPUs love parallel work).

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) were later improvements that helped with the vanishing gradient problem using 'gates' that control what to remember and what to forget. But they still process one step at a time, so the parallelization problem remained -- and this is exactly the gap that Transformers were built to close.

Where RNN is/was used

- Language modeling and text generation (before Transformers took over)
- Speech recognition
- Time-series forecasting
- Machine translation (older systems, pre-Transformer)

PART 2: TRANSFORMERS -- FULL ARCHITECTURE

5. Why Transformers Were Invented

By 2017, RNNs (and their improved versions, LSTM/GRU) were the standard way to handle language tasks, but they had two big problems: they were slow to train because they had to process words one at a time, and they still struggled to connect words that were far apart in a long sentence.

In 2017, a team at Google published a paper called 'Attention Is All You Need'. It introduced the Transformer -- a completely new architecture that removed recurrence (the step-by-step processing of RNNs) entirely, and replaced it with a mechanism called self-attention. This let the model look at all words in a sentence at the same time, and directly learn which words are related to which other words, no matter how far apart they are.

Note: The name of the original paper is literally the core idea: attention is all the model needs -- no recurrence, no convolution required.

The two problems Transformers solved

- Parallelization -- since there's no step-by-step dependency, all words in a sentence can be processed at the same time, making training dramatically faster on GPUs.
- Long-range dependency -- self-attention lets any word directly 'look at' any other word in a single step, so connecting information from far apart in a sentence is no longer a struggle.

6. What is a Transformer? -- The Big Picture

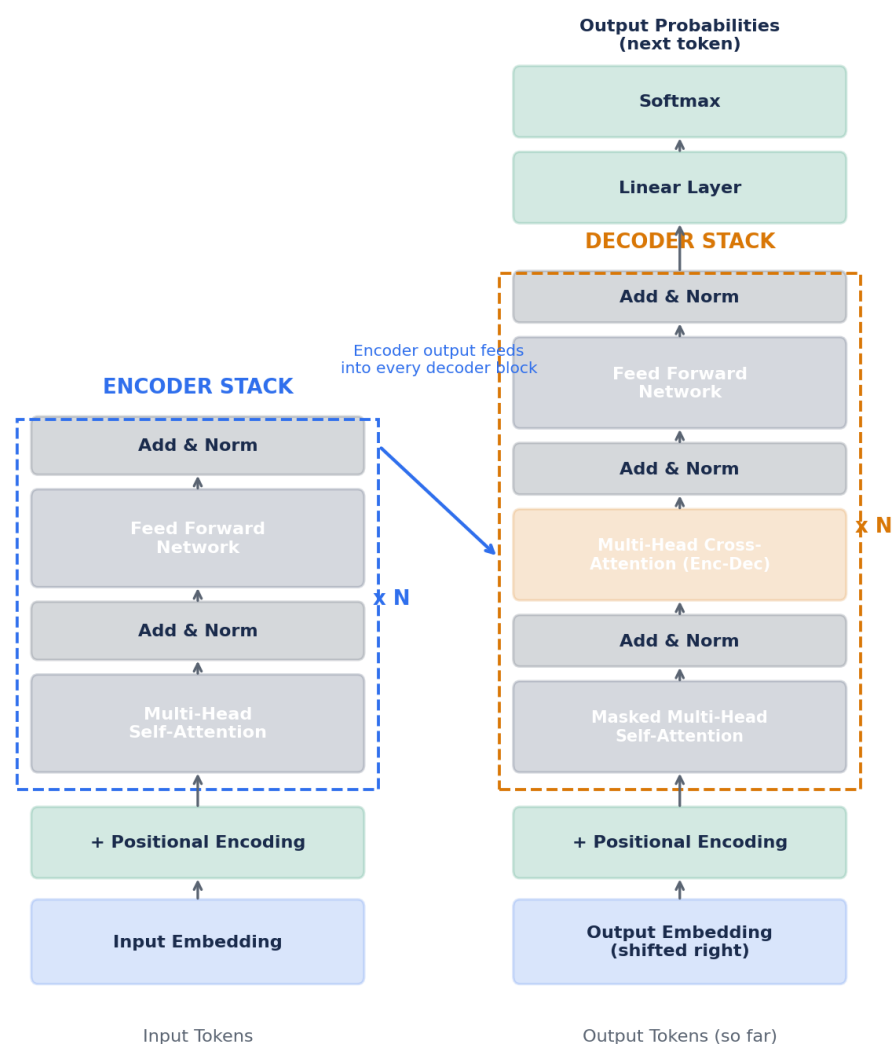
A Transformer is a neural network architecture built almost entirely out of a mechanism called self-attention, combined with simple feed-forward layers. It was originally designed for machine translation (translating one language to another), and its overall design has two main parts: an Encoder and a Decoder.

- Encoder -- reads the input sentence (e.g. an English sentence) and builds a rich understanding of it.
- Decoder -- uses that understanding to generate the output sentence (e.g. the French translation), one word at a time.

Many modern models only use one half of this design. BERT, for example, uses only the Encoder (great for understanding text). GPT uses only the Decoder (great for generating text). But understanding the full Encoder-Decoder design first makes both of those much easier to understand later.

7. The Full Transformer Architecture

Here is the complete picture. Don't worry about understanding every box right away -- every one of these pieces is explained in detail in the sections that follow. Come back to this diagram after reading each section to see how it all fits together.



The full Transformer architecture: Encoder stack on the left, Decoder stack on the right.

Notice the 'x N' label on both stacks. In the original paper, $N = 6$, meaning the encoder block (self-attention + feed-forward) is repeated 6 times in a row, and the same for the decoder block. Each repeat lets the model build a deeper and richer understanding of the input.

8. Input Embeddings

A neural network cannot understand words directly -- it only understands numbers. The first step of a Transformer is to convert every word (technically, every 'token', which can be a whole word or a piece of a word) into a list of numbers called a vector. This is called an embedding.

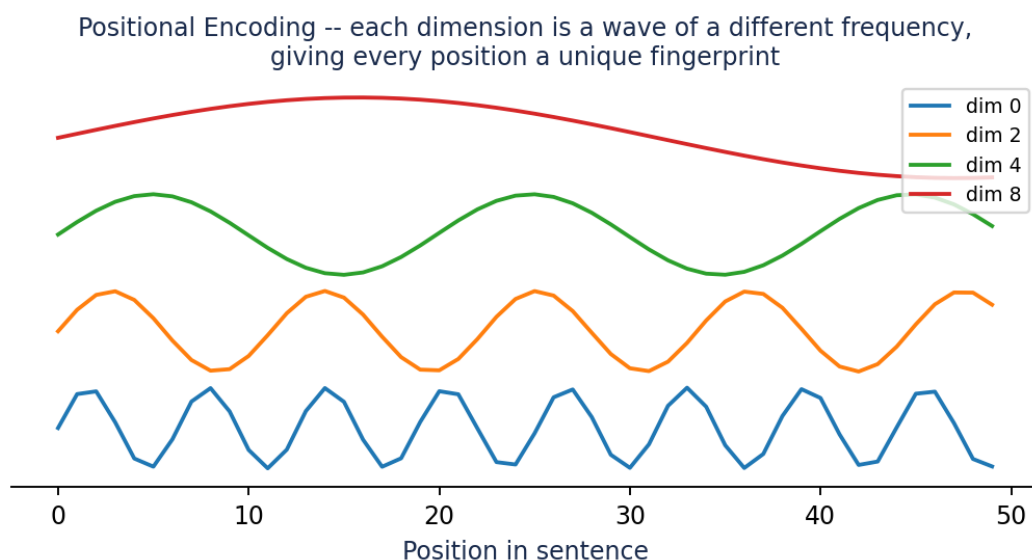
These embeddings are not random -- they are learned during training so that words with similar meanings end up with similar vectors. For example, the vectors for 'king' and 'queen' end up closer to each other than 'king' and 'banana'.

Note: Simple analogy: think of an embedding as a word's address on a giant map, where words with similar meaning live in the same neighborhood.

9. Positional Encoding

Here's a subtle but important problem: self-attention (which we'll cover next) looks at all words at once, with no built-in sense of order. But 'the dog bit the man' and 'the man bit the dog' use the exact same words -- only the order is different, and the meaning is completely different! So the Transformer needs a way to tell the model where each word sits in the sentence.

The solution is Positional Encoding: a set of numbers, generated using sine and cosine wave patterns, that is added to each word's embedding. Each position in the sentence gets a unique combination of wave values, giving the model a way to know 'this word is 1st, this one is 2nd,' and so on -- without needing recurrence like an RNN.



Positional encoding values across different dimensions -- each dimension oscillates at a different frequency, giving every position a unique signature.

Note: You don't need to memorize the exact sine/cosine formula for an interview -- just understand why positional encoding is needed (attention has no built-in order) and the general idea of how it's generated (wave patterns of different frequencies).

10. Self-Attention -- The Heart of the Transformer

This is the single most important idea in the entire Transformer. Self-attention lets every word in a sentence look at every other word, and decide how much attention

(focus) to pay to each one, in order to understand its own meaning better in context.

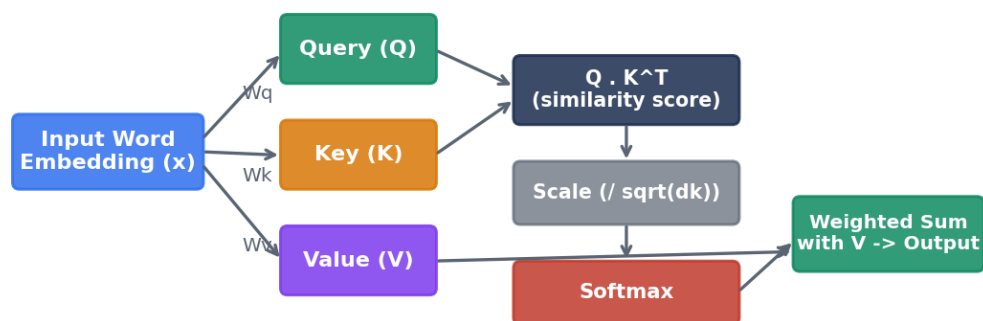
Example: in the sentence 'The animal didn't cross the street because it was too tired', the word 'it' could refer to 'the animal' or 'the street'. Self-attention allows the model to look at the whole sentence and learn that 'it' should pay strong attention to 'the animal' here.

Query, Key, and Value

To do this, every word's embedding is turned into three different vectors, using three separate learned weight matrices:

- Query (Q) -- represents 'what am I looking for?' for this word.
- Key (K) -- represents 'what do I contain?' for every word, used to be matched against queries.
- Value (V) -- represents 'what information do I actually offer?' if I am chosen as relevant.

You can think of it like a search engine: the Query is your search text, the Keys are the titles of all the documents, and the Values are the actual content of those documents. The engine matches your query against every key, and returns a mix of content weighted by how well each key matched.



$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

This lets every word look at every other word and decide how much attention to pay to each one.

How Query, Key, and Value combine to produce the self-attention output for one word.

The formula, explained simply

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

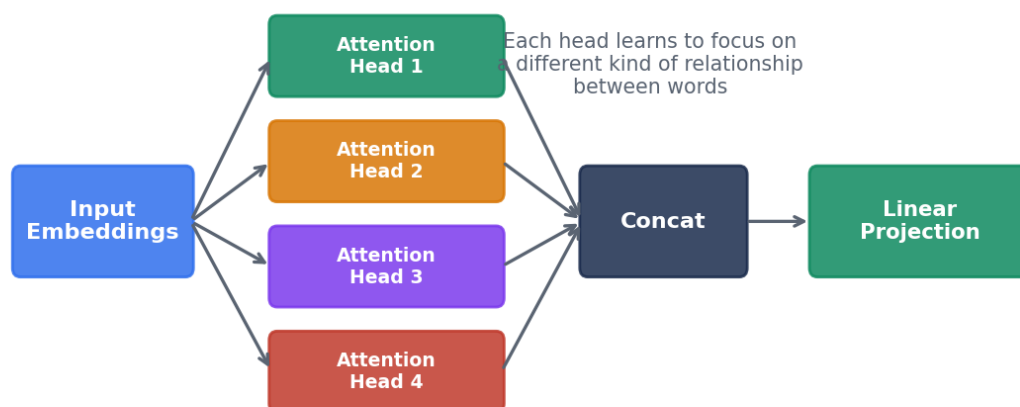
1. $Q \cdot K^T$ -- multiply every word's Query against every word's Key. This gives a raw similarity score between every pair of words.

2. $1/\sqrt{d_k}$ -- divide by the square root of the key vector's dimension. This just keeps the numbers from getting too large, which keeps training stable.
3. softmax -- turns the raw scores into probabilities that add up to 1, so the model can express 'pay 70% attention to word A, 20% to word B, 10% to word C.'
4. Multiply by V -- use those attention weights to take a weighted mix of every word's Value vector. This weighted mix is the final output for that word.

11. Multi-Head Attention

One single attention calculation can only really focus on one type of relationship at a time. But language has many kinds of relationships happening simultaneously -- grammar (subject-verb), meaning (synonyms), position (nearby words), and more.

Multi-Head Attention solves this by running several self-attention calculations in parallel (called 'heads'), each with its own separate Q, K, V weight matrices. Each head can learn to specialize in a different kind of relationship. Their outputs are then combined (concatenated) and passed through one more linear layer.



Multi-Head Attention runs several attention 'heads' in parallel, then combines their outputs.

Note: The original Transformer used 8 heads. More heads let the model capture more different kinds of relationships between words at once, at the cost of more computation.

12. Add & Norm -- Residual Connections + Layer Normalization

You'll notice 'Add & Norm' appears after every major block in the architecture diagram. This is actually two simple but important ideas combined:

- Add (Residual Connection) -- the input to a layer is added back to its output (output = Layer(input) + input). This gives the network a 'shortcut' path, which makes it much easier to train very deep networks (with many stacked layers) without the signal getting lost.
- Norm (Layer Normalization) -- rescales the numbers within each layer to keep them in a stable, consistent range. This helps training converge faster and more reliably.

Note: Simple analogy for the residual connection: imagine editing an essay. Instead of rewriting the whole essay from scratch at every step, you keep the original and add small improvements on top. That's much easier than starting over each time -- and it's exactly what a residual connection lets the network do.

13. The Feed-Forward Network (FFN)

After the attention step, each word's vector (now enriched with context from other words) is passed through a small, simple neural network -- two linear layers with a non-linear activation (usually ReLU) in between. This is applied to each word's vector independently (the same tiny network is reused for every word position).

$$\text{FFN}(x) = \max(0, x \cdot W1 + b1) \cdot W2 + b2$$

Where attention lets words exchange information with each other, the feed-forward layer gives the model extra capacity to transform and process that information further, word by word.

14. Putting It Together: The Encoder Block

One full Encoder block does the following, in order:

1. Multi-Head Self-Attention -- every word looks at every other word in the input sentence and gathers relevant context.
2. Add & Norm -- the residual connection and normalization are applied.
3. Feed-Forward Network -- each word's vector is further transformed.
4. Add & Norm -- again, residual connection and normalization.

This whole block is stacked N times (6 in the original paper). Each stacked layer lets the model build a progressively deeper understanding of the input sentence. The final output of the Encoder stack is a set of vectors -- one per input word -- that richly encode both the word's own meaning and its context within the sentence.

15. Putting It Together: The Decoder Block

The Decoder generates the output sentence one word at a time, and its block has one extra step compared to the Encoder:

1. Masked Multi-Head Self-Attention -- the decoder looks at the words it has generated so far. It is 'masked' so that a word cannot look ahead at words that haven't been generated yet -- this is essential, since during generation, future words simply don't exist yet.
2. Add & Norm.
3. Multi-Head Cross-Attention -- this is where the Encoder and Decoder connect. Here, the Query comes from the decoder (what have I generated so far?), while the Key and Value come from the Encoder's output (what does the input sentence mean?). This lets the decoder pull in relevant information from the original input sentence while generating each new word.
4. Add & Norm.
5. Feed-Forward Network.
6. Add & Norm.

Note: Masking is what makes training efficient: the model can be shown the entire correct output sentence at once during training, but the mask stops it from 'cheating' by looking at the answer before it's supposed to generate it.

16. The Final Output Layer

After passing through all N decoder blocks, the final vector for each output position goes through two more steps:

- Linear Layer -- projects the vector into a much larger vector, with one number for every word in the model's vocabulary (which can be tens of thousands of words).
- Softmax -- converts those numbers into probabilities that add up to 1, so the model can say things like 'there's a 60% chance the next word is dog, 30% chance it's cat, 10% chance it's something else.'

The word with the highest probability (or one sampled according to the probabilities, depending on the generation strategy) is chosen as the output, and the process repeats to generate the next word.

17. How Training Works

During training, the model is shown pairs of correct input and output sentences (for example, an English sentence and its correct French translation). It makes predictions and compares them to the actual correct words using a loss function (typically cross-entropy loss), which measures how far off the predicted probabilities were.

- Teacher Forcing -- during training, the decoder is fed the actual correct previous words (not its own possibly-wrong guesses), which makes training faster and more stable.

- Backpropagation -- the error is used to compute how much every weight in the entire network (encoder and decoder, all attention heads, all FFN layers) contributed to the mistake, and every weight is adjusted slightly to reduce future error.
- This process repeats over huge amounts of text data, often for millions of steps, gradually improving the model.

18. Why Transformers Are So Powerful

- Parallelization -- unlike RNNs, all words are processed simultaneously during training, making full use of modern GPU hardware and dramatically cutting training time.
- Long-range dependency -- self-attention connects any two words in a single step, no matter how far apart they are in the sentence, avoiding the vanishing gradient problem RNNs suffered from.
- Scalability -- Transformers keep getting better as you make them bigger (more layers, more parameters) and feed them more data -- this is the foundation of today's large language models.
- Transfer learning -- a Transformer can be pre-trained once on huge amounts of general text, then fine-tuned cheaply for many different specific tasks.

19. Famous Transformer-Based Models

- BERT (Bidirectional Encoder Representations from Transformers) -- uses only the Encoder. Reads the whole sentence at once (both directions), making it great at understanding text (search, classification, sentiment analysis).
- GPT (Generative Pre-trained Transformer) -- uses only the Decoder. Generates text one word at a time, reading only what came before -- this is the family behind ChatGPT-style models.
- T5 (Text-to-Text Transfer Transformer) -- uses the full Encoder-Decoder design, and treats every NLP task (translation, summarization, question answering) as a 'convert this input text to that output text' problem.
- Vision Transformer (ViT) -- applies the same Transformer idea to images, by splitting an image into small patches and treating each patch like a 'word' in a sentence.

20. Applications of Transformers

- Machine translation (Google Translate and similar systems)
- Chatbots and conversational AI (like the assistant you're reading this from)
- Text summarization
- Question answering and search
- Code generation and code completion
- Image classification and generation (Vision Transformers, and models like DALL-E)

- Speech recognition and generation
- Agentic AI systems -- tool-using agents, RAG pipelines, and multi-agent systems are almost all built on top of Transformer-based language models today.

21. Quick Revision Sheet

- ANN -- basic layered network, no memory, no spatial awareness. Good for simple/tabular data.
- CNN -- uses filters that slide over data, great for images, detects local patterns like edges and shapes.
- RNN -- processes sequences step by step with a hidden memory state, but slow (no parallelization) and struggles with long sequences.
- Transformer -- removes recurrence entirely, uses self-attention so every word can directly look at every other word, in parallel.
- Self-Attention -- Query, Key, Value; $\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$; lets each word gather context from the whole sentence.
- Multi-Head Attention -- runs several attention calculations in parallel, each focusing on different relationships.
- Positional Encoding -- adds sine/cosine-based position information, since attention alone has no sense of word order.
- Add & Norm -- residual connection (shortcut path) + layer normalization (stable training).
- Feed-Forward Network -- a small 2-layer network applied to each word's vector independently.
- Encoder -- understands the input; Decoder -- generates the output, using masked self-attention plus cross-attention into the encoder.
- BERT = Encoder-only, GPT = Decoder-only, T5 = full Encoder-Decoder, ViT = Transformer applied to images.

22. Simple Glossary

- Token -- a small piece of text (a word or part of a word) that the model treats as one unit.
- Embedding -- a list of numbers (a vector) representing the meaning of a token.
- Vector -- just a list of numbers, used to represent things mathematically.
- Weight -- a number the network adjusts during training; the network's 'learned knowledge' lives in its weights.
- Loss function -- a number that measures how wrong the model's prediction was.
- Backpropagation -- the algorithm used to figure out how to adjust every weight to reduce the loss.
- Softmax -- converts a list of numbers into probabilities that add up to 1.

- Parameter -- another word for a weight (or bias) in the network; 'a 7-billion-parameter model' means it has 7 billion of these learnable numbers.
- Fine-tuning -- taking an already-trained model and training it a little more on a specific, smaller dataset for a specific task.
- Inference -- using an already-trained model to make a prediction (as opposed to training it).

22a. Tokenization -- How Text Becomes Tokens

Before a sentence can even reach the Input Embedding step, it has to be broken into tokens -- the small units the model actually works with. This step is called tokenization, and it happens before training even begins.

A simple approach would be to treat every whole word as one token. But this creates a huge vocabulary (English alone has hundreds of thousands of words), and the model still wouldn't know what to do with a brand new word it never saw during training, like a typo or a made-up product name.

Subword tokenization

Most modern Transformers instead use subword tokenization -- breaking words into smaller reusable pieces. Common words stay as a single token, but rare or unfamiliar words get split into familiar chunks. For example, the word 'unbelievable' might be split into 'un', 'believ', and 'able' -- three tokens the model has seen many times before, even if it has never seen this exact word.

- BPE (Byte Pair Encoding) -- starts with individual characters and repeatedly merges the most frequently occurring pairs into new tokens, building up a vocabulary of common subwords. Used by GPT-family models.
- WordPiece -- a very similar idea to BPE, with a slightly different merging rule based on likelihood rather than raw frequency. Used by BERT.
- SentencePiece -- treats the text as a raw stream (including spaces) rather than pre-splitting on whitespace first, which makes it work well across many different languages, including ones that don't use spaces between words.

Note: This is why you'll often hear model limits described in 'tokens', not 'words' -- roughly, 1 token is about 0.75 of an English word on average, though this varies by tokenizer and language.

22b. Model Size -- Parameters, d_model, and Scaling

When people describe a model as having '7 billion parameters' or '175 billion parameters', they mean the total count of learnable weights across every layer -- every Query/Key/Value projection, every feed-forward layer, every layer norm -- added up across all N encoder/decoder blocks.

- `d_model` -- the size of the vector used to represent each token throughout the network (512 in the original paper; much larger, like 4096 or more, in modern large language models).
- Number of layers (N) -- how many encoder/decoder blocks are stacked. More layers generally means the model can build more complex, higher-level understanding.
- Number of attention heads -- how many parallel attention calculations happen inside each Multi-Head Attention block.
- Feed-forward dimension -- the size of the hidden layer inside each block's Feed-Forward Network, usually several times larger than `d_model`.

A key discovery behind today's large language models is that Transformers follow fairly predictable 'scaling laws' -- as you increase model size, training data, and compute together, performance keeps improving in a fairly smooth, predictable way. This is the main reason the field has pushed toward bigger and bigger models over the last few years.

PART 3: COMPARISON, WORKED EXAMPLE & INTERVIEW PREP

23. ANN vs CNN vs RNN vs Transformer -- Side by Side

Before moving on, it helps to see all four architectures compared directly. This table is also a very common way interview questions are framed -- 'when would you use X instead of Y?'

Architecture	Best For	Handles Sequence?	Main Weakness
ANN	Simple/tabular data, basic classification	No	No spatial or sequence awareness
CNN	Images, grid-like data	No (spatial, not sequential)	Not designed for long sequences of text
RNN / LSTM	Sequences (text, speech, time-series)	Yes, step by step	Slow (no parallelization), struggles with long-range dependencies

Architecture	Best For	Handles Sequence?	Main Weakness
Transformer	Sequences, and now images/audio too	Yes, all at once via attention	Needs a lot of data and compute to train well

24. Worked Example: Self-Attention With Real Numbers

Reading the formula is one thing -- seeing it work on tiny numbers makes it click. Let's say we have a mini sentence with just 2 words, and (to keep the numbers simple) each word's Query, Key, and Value are just 2-dimensional vectors instead of the usual 512 or more.

- Word 1 ('cat'): $Q1 = [1, 0]$, $K1 = [1, 0]$, $V1 = [1, 2]$
- Word 2 ('sat'): $Q2 = [0, 1]$, $K2 = [0, 1]$, $V2 = [3, 1]$

Step 1 -- Compute raw attention scores for word 1 ('cat') using its Query against every Key:

- Score with Key1: $Q1 \cdot K1 = (1 \times 1 + 0 \times 0) = 1$
- Score with Key2: $Q1 \cdot K2 = (1 \times 0 + 0 \times 1) = 0$

Step 2 -- Scale by $\sqrt{d_k}$. Here $d_k = 2$, so $\sqrt{2}$ is about 1.41:

- Scaled score 1: $1 / 1.41 = 0.71$
- Scaled score 2: $0 / 1.41 = 0.0$

Step 3 -- Apply softmax to turn these into probabilities that add up to 1 (approximately):

- $\text{Softmax}([0.71, 0.0]) = \text{approximately } [0.67, 0.33]$
- This means: when processing 'cat', pay about 67% attention to itself, and 33% attention to 'sat'.

Step 4 -- Use these weights to combine the Value vectors:

$$\text{Output}(\text{cat}) = 0.67 \times V1 + 0.33 \times V2 = 0.67 \times [1, 2] + 0.33 \times [3, 1] = [1.66, 1.67]$$

That final vector, $[1.66, 1.67]$, is the new representation of the word 'cat' -- no longer just its own meaning, but a blend that also carries a bit of context from 'sat'. The exact same process happens for every word, for every attention head, at every layer -- just with much bigger vectors (usually 512 or 768 dimensions) and many more words.

Note: This is the whole trick of self-attention: turn 'how related are these two words?' into a number, turn that number into a percentage using softmax, and use that percentage to blend information together.

25. Frequently Asked Interview Questions

Q: What problem does self-attention solve that RNNs couldn't?

A: Self-attention lets every word directly connect to every other word in a single step, regardless of distance, and lets all words be processed in parallel -- solving both the long-range dependency problem and the slow, sequential training of RNNs.

Q: Why do we divide by $\sqrt{d_k}$ in the attention formula?

A: Without scaling, the dot products $Q \cdot K^T$ can become very large when the vector dimension is large, pushing softmax into regions with extremely small gradients. Dividing by $\sqrt{d_k}$ keeps the values in a more stable range, which makes training more reliable.

Q: Why does the Transformer need Positional Encoding?

A: Self-attention treats the input as a set (an unordered collection), with no built-in idea of which word came first. Positional encoding injects order information directly into each word's embedding so the model can tell the words apart by position.

Q: What is the difference between Self-Attention and Cross-Attention?

A: In Self-Attention, the Query, Key, and Value all come from the same sequence (a word attending to other words in the same sentence). In Cross-Attention (used in the decoder), the Query comes from the decoder's current sequence, while the Key and Value come from the encoder's output -- letting the decoder pull information from the input sentence.

Q: Why is the decoder's self-attention 'masked'?

A: Masking blocks each position from looking at future positions. This is necessary because at generation time, the model produces words one at a time and genuinely cannot see future words -- masking makes training consistent with that real generation-time constraint.

Q: What is the purpose of residual (skip) connections?

A: They let the input of a layer bypass the layer and be added directly to its output. This creates a shortcut path for gradients during backpropagation, making it possible to train very deep networks (many stacked layers) without the training signal vanishing.

Q: Why use Layer Normalization instead of Batch Normalization in Transformers?

A: Layer Normalization normalizes across the features of a single example, independent of batch size or sequence length, which fits variable-length sequences much better than Batch Normalization, which depends on statistics across the whole batch.

Q: What's the difference between BERT and GPT at an architecture level?

A: BERT uses only the Transformer Encoder and looks at the entire sentence in both directions at once (bidirectional) -- great for understanding tasks. GPT uses only the Transformer Decoder and looks only at previous words (unidirectional/causal) -- great for generating text one word at a time.

Q: What does 'attention head' actually mean?

A: It's one independent self-attention calculation with its own learned Query/Key/Value weight matrices. Multiple heads run in parallel so the model can capture several different kinds of relationships between words at the same time, and their results are concatenated together.

Q: Why are Transformers easier to parallelize than RNNs during training?

A: An RNN must compute hidden state $h(t)$ before it can compute $h(t+1)$, creating a strict sequential dependency. A Transformer's self-attention computes relationships between all word pairs simultaneously using matrix multiplication, which GPUs can execute in parallel.

Q: What is the time complexity concern with self-attention on long sequences?

A: Self-attention computes a score between every pair of words, so its cost grows quadratically with sequence length ($O(n^2)$). This is why very long documents are expensive to process, and why techniques like sparse attention or sliding-window attention exist for long-context models.

Q: What is teacher forcing, and why is it used during training?

A: During training, the decoder is fed the actual correct previous output tokens (from the training data) rather than its own predictions, even if its own prediction was wrong. This keeps training stable and fast, though it can create a small mismatch with real inference, where the model must rely on its own previous outputs.

Q: What does 'pre-trained' mean in the context of models like BERT and GPT?

A: The model is first trained on a huge amount of general text data (with no specific task in mind, just learning language patterns), and only afterward adapted (fine-tuned) to a specific task with a smaller, task-specific dataset -- reusing the general language understanding it already built.

Q: Why is the Transformer described as removing 'recurrence and convolution'?

A: The original 'Attention Is All You Need' paper showed that neither RNN-style recurrence nor CNN-style convolution is required to model sequences well -- self-attention alone, combined with simple feed-forward layers, was enough, and it trained faster and performed better.

Q: In an agentic AI system, where does the Transformer actually fit in?

A: The core language model powering the agent (deciding what to say, which tool to call, and how to reason about a task) is almost always a Transformer-based LLM. Everything discussed elsewhere in agent design -- memory, tools, RAG, multi-agent orchestration -- is built as a system around that core Transformer model.

26. Final Word

If you remember nothing else from this guide, remember this: a Transformer's entire job is to turn 'how related is this word to every other word?' into a number, and use those numbers to blend information together -- repeated across many layers and many attention heads, in parallel, with position information added in so word order isn't lost. Everything else in the architecture (residual connections, normalization, feed-forward layers) exists to make that core idea trainable, stable, and deep.

Note: Good luck with your interview preparation! Re-read Part 2 a second time after finishing the whole guide once -- architecture concepts like this usually click much better on a second pass.